

GSoC 2026 Proposal

Project Title: AI-based Note Categorisation and Tagging

Mentor: HahaBill and Shikuz

Organization: Joplin

Project Size: Large (350 hours)

1. About me and my experiences

Name: Harsh Gupta

University: Harcourt Butler Technical University (HBTU), Kanpur, India

Degree: Bachelor of Technology

Year: 3rd year

Github: [Harsh16gupta](#)

Twitter: [Harsh16Gupta](#)

LinkedIn: [Harsh Gupta](#)

Email Address: harsh16official@gmail.com

Joplin Intro post: [Introducing Harsh16gupta](#)

TimeZone: Indian Standard Time (UTC +5:30)

I was introduced to programming in high school, and since then I have really enjoyed solving problems through code and building useful software. While working on personal and collaborative projects has been a great learning experience, open source has given me the opportunity to contribute to large, real-world codebases used by many people.

1.1 Joplin Contributions

PR Number	Description	Status
#14591	Auto-scroll to selected note from 'Go to Anything' search results	Merged
#14503	Add new option to disable the Joplin icon for internal note links	Merged
#14474	Copying from markdown preview including theme background colour	Merged
#14529	Translate Find and Replace dialog in Rich Text editor	Merged
#14423	Prevent 4th backtick when closing fenced code block	Merged
#14410	Added video tutorials to documentation pages	Merged
#14561	Added the pdf viewer for the Rich text editor	Closed
#14767	ABC Sheet Music rendering out of bound	Merged
#14749	Fixed Custom Dictionary.txt being saved to wrong directory	Merged
#14973	Inline computed styles when copying from the Markdown preview pane	Merged
#14991	Fix inline formatting with trailing/leading whitespace	Merged

1.2 Past Experience in Software Development

GRS Worker: (Freelanced Project)

Designed and developed a full website in a 2-member team, handling system architecture, responsive UI, and backend integration. ([Live](#) | [GitHub](#))

Chess.in: Real-Time Online Chess Platform

Built a real-time online chess platform with WebSocket-based live gameplay, synchronized state updates, and in-game chat. ([GitHub](#))

See2Say: AI Vision-to-Speech Platform

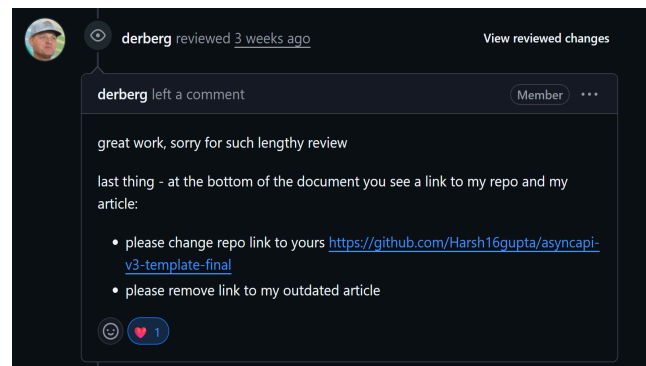
Developed an AI pipeline that converts video frames into narrated audio using OpenCV, BLIP captioning, Gemini summarization, and gTTS. ([GitHub](#))

1.3 Open-Source Experience

My first open-source contribution was to the AsyncAPI Initiative, where I mainly worked on the **AsyncAPI Generator** project. I contributed for about one and a half months and worked on several issues, successfully **merging 18 pull requests** into the generator repository.

Some of my key contribution include:

- **Refactoring the Python WebSocket send helpers** and client send methods from asynchronous to synchronous execution while preserving behavior and error handling. (PR [#1918](#))
- Updated the AsyncAPI Python Template tutorial to fully support AsyncAPI v3, migrating examples from v2 to v3 and updating the template and generator configuration to use the new v3 Parser API . The contribution was **appreciated by the maintainer**, who asked me to replace his repo link with my implementation, which is now referenced in the official [documentation](#). (PR [#1826](#))



1.4 AI Disclosure

I used AI mostly to clean up my grammar and put my thoughts into clearer sentences. I did the research myself. I read through forum discussions, ran actual tests in the plugin sandbox, and figured out what worked and what didn't through real trial and error. A lot of the technical decisions in this proposal came directly from that process.

2. Project Summary

When I started using Joplin for note taking, I imported my notes from onenote and after importing, all my notes were scattered here and there. I had to manually arrange my notes.

This project fixes that by using semantic embeddings to understand what notes are really about, clustering related notes together automatically, and then suggesting tags and notebook structures to the user based on those clusters. It also detects notes that haven't been touched in a long time and flags them as potential archive candidates.

I will create a plugin that shows the user the clusters, tells you what tags it thinks make sense, and asks if you want to apply them.

Why a Plugin Instead of an External Application

While working on this idea, one of the first decisions I had to make was whether the project should be built as a **plugin inside Joplin** or as a **separate external application** that connects to Joplin.

After exploring both possibilities, I chose it to be a plugin because

- **Direct access to notes** :- A plugin can use Joplin's plugin API to access the user's notes directly, which simplifies indexing and processing the note collection.
- **Simpler setup** :- Users can install the plugin directly from the Joplin plugin ecosystem without needing to configure external services.
- **Optional by design** :- Some users on the forum have made it clear they don't want any AI in Joplin. Since this is a plugin, users who don't want it simply don't install it. It doesn't touch the core app.

3. Technical Approach

3.1 JavaScript Only

This is the most important decision in the whole proposal, and it came directly from my own [forum discussion](#) and community feedback.

My original plan (Python subprocess)

When I started researching this project, I realized that 384-dimensional embeddings are hard to cluster directly. Everything starts to look equally distant in very high dimensions. The standard

solution in [research](#) is [UMAP](#) for dimension reduction, followed by [HDBSCAN](#) for density-based clustering. Both work extremely well in Python.

I also discovered two problems with JavaScript. The [umap-js](#) library readme openly states that spectral initialization, a key part of the algorithm, is not fully implemented and HDBSCAN has no usable [JavaScript port](#).

So I proposed using a Python subprocess in my [forum post](#), the plugin would stay in TypeScript for all UI and Joplin API work, but would start a small Python process, send note data via stdin, run clustering, and return results with stdout. I also shared a benchmark showing how much faster the UMAP + HDBSCAN pipeline runs compared to alternatives.

The feedback: Laurent responded that the project should [really avoid this approach](#) and asked whether more recent JS libraries exist. Daeraxa added that for professional users, corporate IT would never allow an extra Python installation. HahaBill who ran into similar issues during his own GSoC project confirmed it was difficult.

They were right. Even if the Python results are slightly better, asking users to have Python installed with the correct packages creates a bad experience. Managing the subprocess lifecycle across Windows, macOS, and Linux is its own engineering problem.

What I found (DruidJS as the better UMAP alternative)

After the feedback I looked for JavaScript alternatives. I found [DruidJS](#) a JavaScript dimensionality reduction library that implements [UMAP](#) along with 14 other algorithms (TSNE, PCA, MDS, ISOMAP, and more).

- **DruidJS:** It is actively maintained and was published in an [IEEE paper](#). Unlike umap-js, it does not have the spectral initialization warning.

Why the algorithm is still UMAP

DruidJS is a library which has 14 algorithms. UMAP is the algorithm I pick from it. One important point here is that DruidJS and umap-js are not different approaches. Both implement UMAP. The only difference is that DruidJS is a better and more maintained library to use.

I also looked at other algorithms in DruidJS. [PCA](#) is linear only and can't untangle non-linear cluster shapes that semantic embeddings often form. [TSNE](#) distorts global distances badly, two clusters that look far apart in TSNE might not actually be far apart in the original space. UMAP preserves both local structure (similar notes stay close) and enough global structure (different topics stay separated).

Note: I validated this by building a working demo inside the Joplin plugin sandbox. DruidJS UMAP runs in ~60ms on 25 notes, and ml-kmeans adds ~10ms. The full pipeline

(Transformers.js embedding → UMAP → K-Means → silhouette scoring → display) works end-to-end. GitHub repo link: [testing-embedding-model](#)

Disadvantages of going with Javascript

The main disadvantage is I can't use HDBSCAN, without it I have to use K-Means (it needs you to specify k, and it forces every note into a cluster even if it doesn't really belong to the cluster). HDBSCAN finds the number of clusters on its own and leaves outlier notes unassigned, which is actually useful for detecting archived notes.

But K-Means after UMAP is drastically better than K-Means on raw 384-dimensional embeddings. UMAP does most of the heavy lifting by separating the topic clusters in a low-dimensional space. K-Means can then find them easily and the k-selection problem which was my original reason for rejecting K-Means can be solved properly using silhouette scores, which I have explained in depth in [section 3.5](#).

3.2 Embedding Model

Why MTEB retrieval score is the wrong metric for this project

MTEB ([Massive Text Embedding Benchmark](#)) has 8 different task types: bitext mining, classification, clustering, pair classification, reranking, retrieval, STS, summarisation. The overall leaderboard score averages all 8. A model that is excellent at retrieval but poor at clustering still ranks highly overall. For this project which is about grouping notes into topics only the clustering score matters only so I filtered MTEB specifically for clustering tasks..

Models I looked at and rejected

Before finalising on [BGE-small](#), I went through several other candidates and ruled most out for concrete reasons.

- [MedEmbed-small-v0.1](#) fine-tuned specifically on medical text. I looked into this because it has a very good clustering score but had to reject it. Personal notes are not clinical records. A [model trained](#) on clinical language will cluster notes about cooking, travel, and software development poorly.
- [GIST-small-Embedding-v0](#) has a clustering score (46.7) which is actually slightly higher than BGE-small (46.3). But its zero-shot status on MTEB is marked as 'NA', so its results are not fully reliable (possible data leakage), so I didn't want to use it.
- Several other models had blank benchmark columns. They were not tested on all clustering tasks, so their scores come from partial testing only and are not comparable.

Rank	Model	Zero-shot	Memory Usage	Number of Parameters	Embedding Dim	Max Tokens	Mean (Task)	Mean (Task)	Clustering
1	[NoInstruct-small-Embedding-v0](https://huggingface.co/NoInstruct-small-Embedding-v0)	NA	127	0.033	384	512	46.8	46.8	46.8
2	[GIST-small-Embedding-v0](https://huggingface.co/GIST-small-Embedding-v0)	100%	127	0.033	384	512	46.7	46.7	46.7
3	[bge-small-en-v1.5](https://huggingface.co/BAAI/bge-small-en-v1.5)	100%	127	0.033	512	512	46.3	46.3	46.3
4	[MedEmbed-small-v0.1](https://huggingface.co/MedEmbed-small-v0.1)	100%	127	0.033	384	512	46.22	46.22	46.22
5	[gte-small](https://huggingface.co/thenlper/gte-small)	NA	64	0.033	384	512	46.08	46.08	46.08
6	[GIST-all-MiniLM-L6-v2](https://huggingface.co/GIST-all-MiniLM-L6-v2)	100%	87	0.023	384	512	45.28	45.28	45.28
7	[granite-embedding-30m-english](https://huggingface.co/granite-embedding-30m-english)	83%	58	0.03	384	512	45.76	45.76	45.76
8	[mini-gte](https://huggingface.co/prdev/mini-gte)	100%	253	0.066	768	512		51.55	51.55
9	[slx-v0.1](https://huggingface.co/brahmairesea/slx-v0.1)	NA	87	0.023	384	512	44.53	44.53	44.53
10	[mdbr-leaf-mt](https://huggingface.co/MongoDB/leaf-mt)	100%	86	0.023	1024	512		50.33	50.33

Chart which I downloaded (Applying the filters for clustering)

I first chose [BGE-small-en-v1.5](#) for these specific reasons: clustering score of 46.3 (3rd highest with complete benchmark data), memory usage of 127 MB (comfortable on any modern machine), maximum token limit of 512 (handles most normal Joplin notes chunking will handle the rest), zero-shot status confirmed at 100% (so the benchmark score is trustworthy) and MIT license.

Changes based on the feedback: Bill (mentor for this project) asked me to run the the embedding model(BGE-small-en-v1.5 and all-MiniLM-L6-v2) with ONNX Transformers.js in the plugin environment. Transformer.js v3 within the plugin environment and tested the speed of the embedding model on various notes.

My findings: BGE-small takes roughly twice as long to embed the same notes compared to all-MiniLM-L6-v2. Both models share the same max token limit (512) and output dimension (384), so the quality loss is minimal while the speed gain is very large.

Link : [Forum discussion](#)

GitHub repo link: [testing-embedding-model](#)

Note: I still need to confirm one thing before finalising the embedding model: all-MiniLM-L6-v2 has a 256 token sequence limit, and I want to verify that chunks with CCH prepended stay within that. Until then, BGE-small (512 token limit) avoids any truncation risk. I will design the architecture to be modular so that swapping the model is a one-line change. So that when we want to support multiple languages embedding that can be easily done by changing the model name([discussed](#) with the mentor).

3.3 Note Representation

Before I explain the chunking strategy, I want to explain why it is necessary. BGE-small has a 512-token limit (roughly 380 words). If a note is 1500 tokens long, the model silently ignores everything after token 512. There is no error, no warning. For categorization this is a moderate problem; the main topic is usually in the first few paragraphs, so truncation causes some misclustering but not total failure. The notes that suffer most are ones where the main subject only becomes clear midway through, or notes that cover multiple topics where the second topic falls past token 512.

At first I tried the simplest thing to concatenate the note title and body into one string, embed it, and use that as the note's vector. One note, one embedding. The problem with this approach was that long notes contain multiple topics. When you embed everything as one piece, the vector lands somewhere in the average of all those topics. It doesn't represent any of them cleanly, and the note ends up in the wrong cluster.

So I split each note into chunks of roughly 380 words, with a 50-word overlap between consecutive chunks. The overlap is important without it, a sentence that falls on the boundary between two chunks gets split in half. Each chunk sees half a sentence, which loses the meaning. With 50-word overlap, every sentence appears complete in at least one chunk.

Going from chunks to a single note vector

After each chunk is embedded separately, I average all the chunk vectors into one single note vector. I proposed this approach in the [GSoC AI opportunities forum discussion](#) for categorization (as opposed to chat/retrieval), and it was appreciated by Shikuz (jarvis creator and GSoC mentor).

Title weighting

To calculate the final_vector I was thinking of using title_vector. A good title tells a lot about the whole note in 5–10 words, so it makes sense to give it a bit more weight when calculating what a note means. So my first thought was to give the title_vector 30% of weight.

$$\text{final_vector} = (\text{body_avg_vector} \times 0.7) + (\text{title_vector} \times 0.3)$$

But many notes have titles that don't actually say anything useful: "Untitled", "New Note", a date, or just a single word. Giving a meaningless title 30% weight makes no sense. So after discussing with Laurent on the [forum](#). We came to a conclusion that I need to add a pre-processing step to remove all the generic titles.

After filtering the generic titles, I thought about how to decide how much weight to give to the titles that survive. I came up with two ways:

1. Word count: give more weight to longer titles, less to shorter ones. A 6 word title gets 0.3, and it goes on decreasing. This is simple and fast but this has an issue, what if the title is long and wrong(unrelated to the note body) then it will affect the cluster formed.

2. Cosine similarity: In this I will embed the title separately and compare it to the body_avg_vector. If they're talking about the same thing, **similarity will be high** and I give the **title more weight**. If they're mismatched, similarity drops and the weight reduces automatically. This is a better approach, but it adds one extra embedding call per note(will add 60 more seconds if there are 2000 notes only once at the start).

For short focused notes the body average is already precise, so the title doesn't add much either way. But for longer notes, the kind where the body covers multiple topics and the average vector gets blurry, a good title pulls the final vector toward the actual main topic. On a 2000-note collection, probably 300–400 of those longer notes would benefit noticeably from this. The extra cost is around 60 seconds on the first run, and after that everything is cached so it doesn't repeat.

3.4 The Clustering Problem

Clustering the data from the notes was an important part of the project and it also took most of the work.

Simple K-Means ([K-Means](#) is simple, fast, and available in JS)

My first plan was to embed all notes, run K-Means and get the notes clustered. The problem was it needed us to specify the value of K (the number of clusters) [up front](#). Users don't know how many topic clusters their notes form. Choosing k wrong means either splitting related notes into too many groups or forcing unrelated notes together. Using [ml-kmeans](#) with kmeans++ initialization.

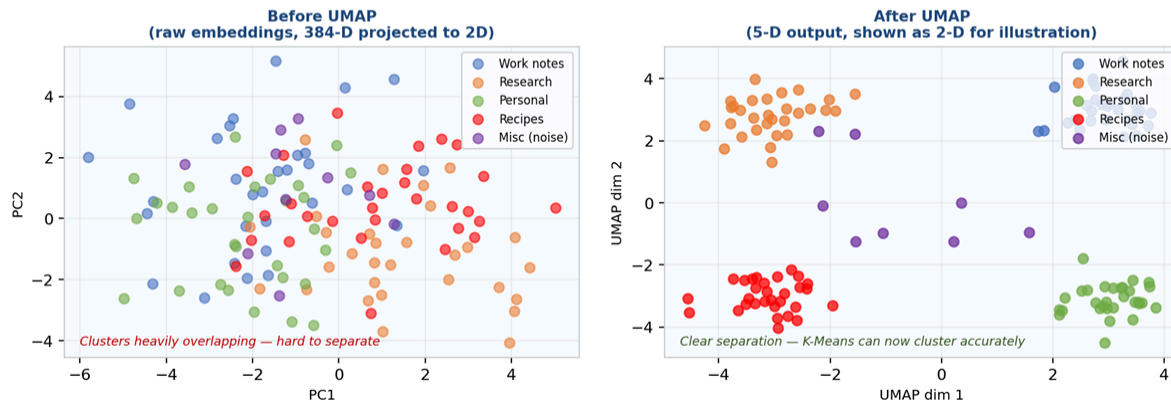
HDBSCAN (It finds the number of clusters automatically and can also handle isolated notes)

It is way better than K-means as it is a density clustering tool but there is no reliable pure JS HDBSCAN library. The Python hdbscan package works brilliantly, but as Laurent noted in the forum, Python subprocesses are something to avoid. I looked for a JS alternative and did not find one that I could trust in production (some are still under development).

Final approach (UMAP + K-Means)

UMAP fixes this. UMAP (via DruidJS) learns a lower-dimensional space where similar notes are packed tightly together and dissimilar notes are pushed apart. The [UMAP clustering documentation](#) shows: HDBSCAN on raw 50-dimensional data clustered only about 17% of points correctly. After reducing with UMAP first, the same data clustered at ~99%. This is a great improvement and this data help me finalise my decision.

Note: I tested this in my demo [plugin](#). On a 25-note corpus with very diverse content (tech notes, Wikipedia articles, biographies), UMAP + K-Means with silhouette scoring correctly separated short topical notes from long encyclopedic articles, choosing k=2 with a silhouette of 0.283.



3.5 K-Calculation

This calculation helps in finding the value of K. As I said in my forum post: 'I am not using K-Means because there we need to enter how many clusters we want to create.' HahaBill's response was: use K-Means with optimal k calculation. So here is how I will do that properly.

The silhouette score

The silhouette score measures how well a note fits its own cluster compared to the nearest other cluster. For each note, it computes two things:

- a = the average distance from this note to all other notes in its own cluster (how tight the cluster is)
- b = the average distance from this note to all notes in the nearest other cluster (how far apart clusters are)

$$\text{The silhouette score} = (b - a) / \max(a, b) \quad (\text{for one note})$$

It ranges from -1 to +1. A score close to +1 means the note fits its cluster well and is far from other clusters. A score close to 0 means the note is near the boundary. A negative score means the note might actually belong to a different cluster.

The average silhouette score across all notes tells you how good the clustering is for a given value of k. So instead of guessing k, I try many values and pick the one with the highest average silhouette score.

The algorithm:

Step 1: Run UMAP

I will reduce all note vectors using UMAP. This will give me a smaller matrix (like $N \times 5$). It will be

done once per session.

Step 2: Choose k range

Set a range of cluster values: $k = 2$ up to $\sqrt{N}$ (rounded down).

Example:

- 100 notes $\rightarrow k$ up to 10
- 1000 notes $\rightarrow k$ up to 31

Step 3: Try K-means for each k

For each value of k , run K-means using kmeans++ initialization and a fixed seed for reproducibility. Save the cluster assignments for each run.

Step 4: Score each k

For every k , calculate the silhouette score using the UMAP-reduced vectors. This tells how well the clusters are formed.

Step 5: Pick best k

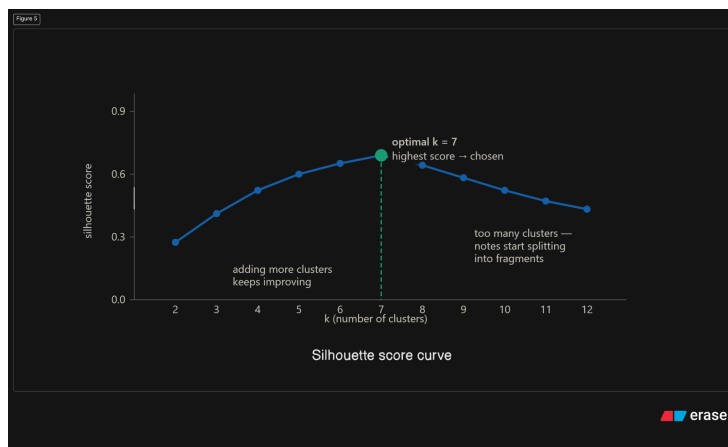
Compare all the scores and choose the k with the **highest value**. This is the most “natural” number of clusters for the data.

Step 6: Final clustering

Run K-means again with this selected k to get clean and stable final clusters.

Step 7: Save results

Store the final k , scores, and cluster assignments in the database, so we don't have to recompute unless notes change.



Why k_range = 2 to $\sqrt{N}$

At first I thought about using a fixed range like $k = 2$ to 20. But that doesn't scale. If someone has 2000 notes, they might have 30 or 40 natural topic groups. A fixed upper limit of 20 would miss them.

$\sqrt{N}$ is a simple way to scale with the number of notes without going too far. As the collection grows, we allow more possible clusters, but still keep the search space manageable.

For example, 100 notes $\rightarrow$ k up to 10,

500 notes $\rightarrow$ up to 22,

2000 notes $\rightarrow$ around 44.

In most cases, the silhouette score peaks much earlier, so we don't actually need to try all values.

Performance

For a 1000-note collection, trying k from 2 to 31 means 30 K-Means runs. K-Means on 1000 points in 5 dimensions is nearly instant each run takes under 100ms. So 30 runs takes about 3 seconds total. That is completely acceptable for a one-time analysis that runs in the background.

I also cache the result. Once the optimal k and cluster assignments are computed, they are stored in sqlite alongside a hash of the note collection. If notes haven't changed significantly, the user gets instant results on re-open. Silhouette calculation only re-runs if a meaningful number of notes have changed.

Edge cases:

- **Fewer than 10 notes**
When there are very few notes I will skip UMAP and the scoring step. Just use k = 2 and show two groups. Small collections don't need complex clustering.
- **All notes are similar**
Silhouette scores will be low for all k values. We pick k=2 anyway and show the user one large cluster with a note that says the collection appears very uniform.
- **Notes are very different**
Silhouette score will peak at a higher k. The algorithm finds it automatically via the $\sqrt{N}$ search.
- **UMAP randomness**
Use a fixed `seed: 42` so the output stays consistent every time.

Why silhouette score method only?

There were some other methods to find the value of k like the [elbow method](#), [gap statistics](#) method and [Davies-Bouldin Index](#) method.

The elbow method was my first thought, in this you plot inertia against k and look for where the curve bends. Simple idea but the problem is detecting that bend automatically in code is

actually hard. The curve is sometimes smooth with no clear elbow, and I couldn't find a reliable way to pick it programmatically without a human looking at the plot.

Gap statistics is more accurate but it builds a random baseline by generating and clustering multiple fake datasets and is way too expensive to run on a user's machine in the background.

Davies-Bouldin is fast and gives a clear number like silhouette does, but it tends to keep rewarding you for splitting clusters further, which pushes k higher than it should be for a note collection where users want broad topic groups.

3.6 UMAP Settings

UMAP has several hyperparameters that can have a significant impact on the resulting embedding. In this notebook we will be covering the four major ones:

- `n_neighbors`
- `min_dist`
- `n_components`
- `metric`

I went through the [notebook](#) and found what can be the best value to each.

- **`n_neighbors = 15`**
This controls how many nearby notes UMAP looks at when building its understanding of the local structure. Too low and it fragments everything into tiny isolated groups. Too high and it starts merging unrelated topics together because it's looking too broadly. 15 is BERTopic's default and works well for collections between 100 and 2000 notes.
- **`n_components = 5`**
This is how many dimensions UMAP reduces down to. I could go lower as 2 or 3 but that loses too much information and clusters start overlapping. I could go higher, but then K-Means starts struggling again because the space gets too big. 5 is the sweet spot that BERTopic also uses for topic clustering.
- **`min_dist = 0.0`**
This controls how tightly UMAP packs similar notes together. When you're making a visualization you want some spread so it looks nice. But for clustering you actually want the opposite notes that are similar should be packed as close as possible so K-Means can find clean boundaries between groups. Setting it to 0 does this.
- **`metric = 'cosine'`**
Text embeddings should be compared by angle, not by size. Two notes can have very different length bodies but still be about the same topic cosine similarity captures that, Euclidean distance does not capture that.

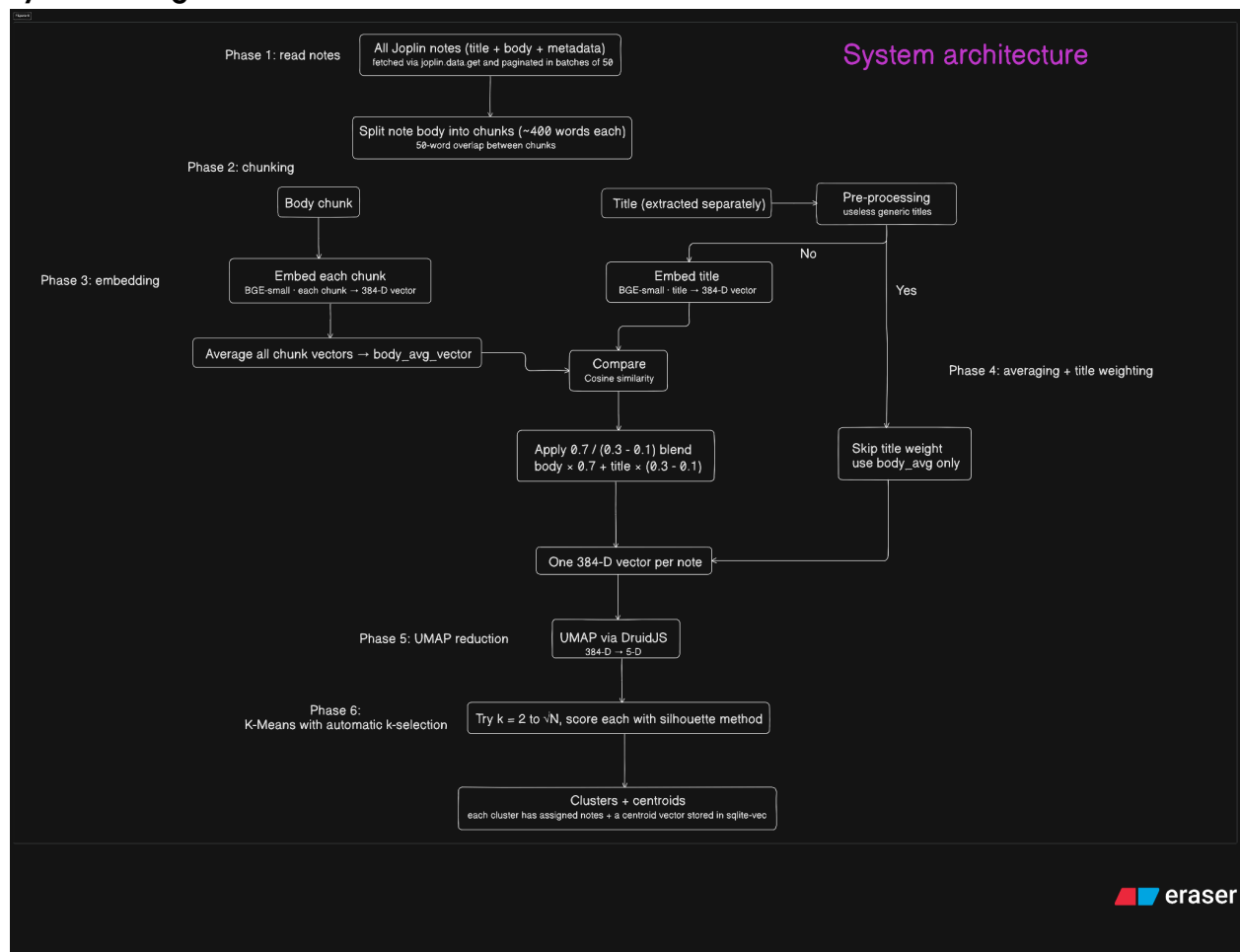
- **seed: 42**

UMAP has some randomness in it, which means if you run it twice on the same data you might get slightly different results. That would be confusing for users, their clusters would shift every time they re-run the analysis. Fixing the seed makes sure the output is the same every time.

I can also let users adjust `n_neighbors` and `n_components` from the settings panel, since someone with 50 notes needs different values than someone with 5000 (will discuss this with the mentor).

Note: These values are based on the [BERTopic parameter guide](#) and the UMAP documentation. These values are not final. I will test UMAP on a large variety of notes then decide a final value.

System design till cluster formation:



3.7 Tag Generation

After clustering, each group needs a name but the system does not know what to call the group. Once clusters are formed and centroids are computed, the plugin generates tag suggestions for each cluster through a three-step pipeline:

Step 1: Extract candidate terms using TF-IDF

I collect all the text from every note in the cluster and treat it as one combined document. Then I run TF-IDF (using the [natural](#) npm library) across all clusters, where each cluster is a document. This helps me find words that appear often in this cluster but not in others. From these, I pick the top 15 important terms.

Before feeding tokens into TF-IDF, I will also generate [bigrams](#) and [trigrams](#) using the natural library's built-in N-gram support. So instead of only scoring individual words, the candidate pool also includes multi-word phrases like "machine learning" or "data science" as single terms. This will give way better tag names.

For deduplication, I will use a simple greedy approach: once a phrase like "machine learning" gets picked as a candidate, both "machine" and "learning" are marked as used and skipped if they show up again individually. This was an edge case I hadn't thought about initially (pointed out in review by the maintainer).

Note: natural npm library works perfectly in the plugin environment. It was used by Bill in the summarisation plugin.

Step 2: Re-rank candidates by centroid similarity

Each candidate term is embedded using the same BGE-small model which I used earlier. I then compute the cosine similarity between each candidate's embedding and the cluster centroid. Terms that are closest to the centroid are chosen, since they better represent what the cluster is actually about, not just frequent words.

Step 3: Compute a confidence score

For each candidate, the confidence score is:

$$\text{confidence} = (0.4 \times \text{tfidf_score_normalized}) + (0.6 \times \text{cosine_similarity_to_centroid})$$

I normalize TF-IDF scores to 0–1 range within each cluster and give slightly more importance to similarity, while TF-IDF still helps capture important terms. The top 3–5 candidates by this combined score are shown to the user. The highest-scoring one is pre-selected as the default suggestion, but the user can pick any of them or type their own.

LLM layer (optional) If the user has configured an API key (OpenAI or Gemini) or a local Ollama instance, the plugin sends only the top 5 TF-IDF keywords per cluster, not the actual note text to the LLM and asks for a clean, human-readable label. The prompt is something like:

"Given these keywords from a group of related notes: [recipe, cooking, ingredients, preparation, dinner]. Suggest a short tag name for this group." The LLM response replaces the default suggestion but the user still confirms before anything is applied.

3.8 Notebook Organization

My first idea was to automatically create a notebook for each cluster and move all notes into it. When I thought more about it, I realized that is not the right approach. Moving notes is destructive. If a user has 500 notes organized their way over two years, and the plugin restructures everything based on what it thinks, that could cause real problems. Also, in Joplin each note belongs to exactly one notebook.

So the plugin only suggests. Each cluster is shown in the UI with:

- The cluster's proposed tag name (editable inline)
- A list of notes in the cluster (clickable to open)
- A button: 'Add this tag to all notes in cluster'
- A button: 'Create new notebook and move notes' (with a warning)
- A Skip option (no action taken)

Nothing happens until the user asks to change the cluster.

Centroid-based notebook assignment for future notes

When a user creates a notebook from a cluster, I store the cluster centroid alongside the notebook record. This helps when new notes are added to Joplin, the plugin can **suggest which existing notebook they likely belong** to by comparing the new note's vector to all stored centroids and finding the nearest one.

Where centroids are stored

When a user creates a notebook from a cluster, I save that cluster's centroid vector (the average of all note vectors in the cluster) in the plugin's vectra database, linked to the notebook's Joplin ID. It's just one row per notebook. If the user later adds more notes to that notebook by hand, the centroid doesn't update automatically (that would mean re-embedding everything in the notebook). Instead, the centroid refreshes when the user clicks 'Re-analyse.'

How the suggestion logic works for new notes When a new note is created or heavily edited, the plugin does this:

1. Embeds the note using the same pipeline (chunk → embed → average).
2. Compare it to every stored notebook centroid using cosine similarity.
3. Finds the highest similarity score.
4. If similarity ≥ 0.65 , it shows a suggestion: "This note looks like it belongs in [Notebook Name]". The user can move the note to the Notebook in one click.

5. If similarity is between 0.45 and 0.65, it shows a softer suggestion: "This note might be related to [Notebook Name]".
6. If similarity < 0.45, shows nothing. The note is probably about something new.

The 0.65 and 0.45 numbers are starting values. I'll test them on real note collections during development and see if the suggestions feel right. These thresholds will also be adjustable in the settings panel.

This process of finding where the new note belongs is K-nearest neighbours (KNN) with $k=1$. It takes the new note, compares it to all notebook centroids, and assigns it to the closest one.

The final decision of whether to approve or ignore the suggestion is still in the hand of the user but it means the plugin will get more useful over time as more notebooks are created and their centroids are known.

3.9 Archive Detection

For notes that have been sitting untouched for a long time and might be clutter. I looked into whether ML would help here, maybe I can train a classifier on which notes users archive vs keep. But that requires labeled training data which I don't have.

How I calculate the staleness score

Each note gets a staleness score between 0 and 1. It's a weighted sum of five signals:

Signal	Weight	How I calculate it
Last edited	0.30	<code>days_since_edit / 365</code> , capped at 1.0. A note not touched for a year or more gets the max score.
Edit count	0.15	<code>1 - min(edit_count, 10) / 10</code> . A note edited only once scores 0.9. A note edited 10+ times scores 0 (note is still active).
Content length	0.10	1.0 if the note is under 100 characters and is not a to-do item, otherwise 0.0. (This will catch stubs and abandoned drafts)
Backlinks	0.15	1.0 if no other note links to this one, otherwise 0.0. If other notes reference it, it's probably useful.

Cluster fit (silhouette)	0.30	1 - $\max(\text{individual_silhouette}, 0)$. If it has a negative silhouette (meaning it doesn't really belong in any cluster), it gets a high score.
-----------------------------	------	---

staleness_score = (0.30 × last_edited) + (0.15 × edit_count) + (0.10 × content_length) + (0.15 × backlinks) + (0.30 × cluster_fit)

Why these weights?

Last-edited and cluster-fit get the most weight because they're the strongest signals. A note that hasn't been touched in a year AND doesn't fit any topic cluster is almost certainly archivable. Edit count and backlinks are medium signals; a note might only have one edit but still be important (like a reference page you wrote once and keep going back to). Content length is the weakest signal because short notes can be intentional (a phone number, an address).

When does a note show up as an archive suggestion? Notes with a staleness score above 0.6 appear in the Archive Suggestions section. I went with 0.6 instead of 0.5 because I'd rather miss a few stale notes than annoy the user by suggesting notes they still care about. This threshold is also adjustable in settings.

What the user sees Each suggested note shows: its title, when it was last edited, its staleness score (as a percentage), and the main reason it showed up (e.g., "Not edited in 14 months" or "Doesn't fit any topic group"). Each note has a checkbox. The user picks which ones to tag as 'archive' and clicks confirm. Nothing happens automatically.

High-scoring notes appear in the 'Archive Suggestions' section. The user sees the last-edited date and a checkbox. Nothing is archived automatically.

3.10 UI Integration

The UI uses a persistent sidebar panel built using [joplin.views.panels](#). The panel has three sections:

- **Clusters section:** Each detected cluster shows as a card with the proposed tag, note count, sample note titles, and action buttons.
- **Archive Suggestions section:** Lists stale notes with their last-edited date and a checkbox.
- **Settings section:** UMAP parameters, embedding provider, LLM provider for optional tag generation, and a 'Re-analyze notes' button.

What happens on first launch When the user opens the panel for the first time, it shows an empty screen with one button: "Analyse my notes." Clicking it starts the full pipeline. A progress bar shows what's happening:

- "Embedding notes... (142/500)"

- "Reducing dimensions..."
- "Finding clusters..."
- "Generating tag suggestions..."

If the user closes the panel while it's still running, the embedding work done so far is saved (it's already in vectra). Next time they open it, the plugin picks up where it left off.

What a cluster card looks like Each cluster shows as a card with:

- A colored dot (each cluster gets its own color so you can tell them apart)
- The suggested tag name in an editable text field (pre-filled with the top suggestion)
- A small label showing how many notes are in the cluster (e.g., "12 notes")
- 3 sample note titles (clickable, opens the note in the editor)
- A label showing how strong the cluster is (e.g., "Strong match" or "Moderate match" based on the average silhouette score)
- Buttons: "Apply tag to all" · "Create notebook" · "Skip"

Error handling If embedding fails for a specific note (say the content is corrupted), the plugin skips it and lists it in a small "Skipped notes" section at the bottom. The rest of the pipeline continues as normal. If UMAP or K-Means fails entirely (which would only happen with really weird data), the panel shows a clear error message with a "Retry" button.

3.11 How Cluster Suggestions Become Real Joplin Actions

Here is the exact sequence of what happens when the user confirms a cluster suggestion.

Creating and assigning a tag

```
import joplin from 'api';
async function createAndAssignTag(tagName: string, noteIds: string[]) {
  // First, check if the tag already exists to avoid duplicates
  let page = 1;
  let allTags: { id: string; title: string }[] = [];
  while (true) {
    const result = await joplin.data.get(['tags'], { fields: ['id', 'title'], page: page });
    allTags = allTags.concat(result.items);
    if (!result.has_more) break;
    page++;
  }

  const existing = allTags.find(t => t.title === tagName);

  // If it does not exist, create it
  const tag = existing ? existing : await joplin.data.post(['tags'], null, { title: tagName });

  // Then assign it to every note in the cluster
  for (const noteId of noteIds) {
    await joplin.data.post(['tags', tag.id, 'notes'], null, { id: noteId });
  }
}
```

Creating a notebook and moving notes

```
async function createNotebookAndMoveNotes(notebookName: string, noteIds: string[]) {
  // Create the notebook
  const folder = await joplin.data.post(['folders'], null, { title: notebookName });

  // Move each note into it
  for (const noteId of noteIds) {
    await joplin.data.put(['notes', noteId], null, { parent_id: folder.id });
  }
}
```

Applying an archive tag

```
async function applyArchiveTag(noteIds: string[]) {
  const archiveTag = await joplin.data.post(['tags'], null, { title: 'archive' });

  for (const noteId of noteIds) {
    await joplin.data.post(['tags', archiveTag.id, 'notes'], null, { id: noteId });
  }
}
```

All of these calls happen only after the user clicks confirm. The plugin never modifies notes, tags, or notebooks without explicit user action. The Joplin API handles persistence and sync once the plugin makes these calls, changes appear across all of the user's synced devices automatically.

What if something fails? Every API call is wrapped in a try-catch. If assigning a tag to one note fails (maybe the note was deleted between analysis and when the user clicked confirm), the plugin logs the error, skips that note, and continues with the rest. After the whole batch is done, any skipped notes are shown to the user so they know what happened.

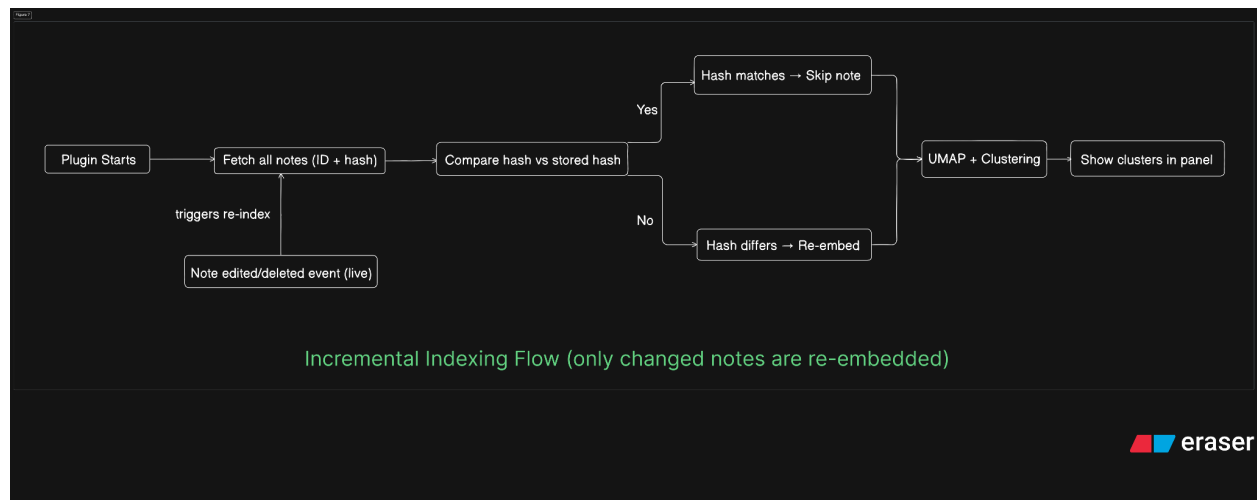
3.12 Incremental Indexing

The first time the plugin runs, it embeds all notes. That could take a few minutes for a large collection. After that, re-doing everything every time would be a bad experience.

So I implement incremental indexing. Every note has a SHA-256 hash of its body stored alongside the embedding in vectra. On startup, the plugin hashes each note and compares it to the stored value. If it matches then it skips. If it differs then it is re-embed. Only modified notes get new embeddings.

I also listen for Joplin note change events. When a note is edited or deleted, the index updates automatically in the background. UMAP and clustering only re-run when the user explicitly clicks 'Re-analyse' or when a meaningful number of notes have changed (thresholded at roughly 5% of the collection). I picked 5% because that means about 1 in 20 notes has changed, which is

enough to shift where cluster boundaries sit. Below that, the existing clustering is still close enough. This threshold will be also adjustable in settings, someone who adds notes in bursts might want to set it higher (like 10%) so re-analysis doesn't kick in too often.



3.13 Shared Infrastructure

I am also ready to make this having a shared infrastructure which has been discussed on the [forum](#). I have also [proposed](#) to have the same average chunk embeddings for idea 3 and idea 2 (Shikuz also [appreciated](#) it). I am fully ready to make the project in such a way that we can easily integrate it with the shared MCP server.

3.14 Privacy

Everything is local by default. Embeddings are computed on-device using ONNX via transformers.js. UMAP and K-Means run entirely in JavaScript. No note content ever leaves the user's machine unless they explicitly configure an LLM API key and even then, only the top cluster keywords (not actual note text) are sent to the LLM.

Embeddings and cluster assignments are stored in the plugin's own data directory and are never synced to Joplin's main database. This matches Joplin's local-first design philosophy.

3.15 Known Challenges

- WASM memory degradation during large batch embedding:** The WebAssembly linear memory grows during inference but never gets released. On BGE-small this means throughput drops from around 47 notes per second at the start of a session to roughly 2 notes per second after processing about 100 notes (this will make it very slow).
 To avoid this I will periodically recycle the worker process during batch embedding. On

every 80–100 notes, I will reinitialise the pipeline object and continue from where it left. And when each batch completes I will write embeddings to vectra, no work is lost on recycling. I will validate the exact recycle interval during the community bonding period.

- **UMAP for very small collections:** UMAP needs `n_neighbors` less than the number of notes. For collections under 20 notes, I'll skip UMAP or reduce `n_neighbors` automatically. For tiny collections, simple clustering on raw embeddings is fine.
- **ONNX/transformers.js in Electron:** Running ML inside Joplin (Electron + Webpack) can have WebAssembly compatibility issues. This is a documented risk from past GSoC projects. I have **already validated this**. The embedding pipeline runs correctly in the plugin environment using a Web Worker with target "web"([repo](#)).
- **K-selection with silhouette:** Trying multiple `k` values adds time. For 1000 notes, trying `k` from 2 to 32 might take 10–20 seconds. I'll cache results so this doesn't re-run unless notes have changed significantly.

4. Implementation

Community Bonding

Will go in depth of plugin creation (I have already made a Table of content [plugin](#) going through the docs and gave it a [new touch](#)). Experiment with UMAP parameters. Discuss scope with the mentor.

Outcome: *Embedding pipeline validated in plugin environment.*

Week 1–2: Infrastructure

Will set up the plugin structure and implement the core data pipeline (reading notes using the Joplin Data API, handling pagination properly, and extracting only required fields). I will implement chunking and embedding of notes, along with SHA-256–based change detection (only modified notes are reprocessed). Embeddings will be stored in vectra.

Outcome: *Notes can be read, chunked, embedded, and stored correctly. The plugin runs without errors*

Week 3–4: UMAP and Note Vectors

Will implement chunk-to-note vector aggregation and introduce title weighting. Along with generic title detection and fallback. I will integrate UMAP using DruidJS and test it on different collection sizes (small, medium, large). I will confirm stable output with fixed seed.

Outcome: complete embedding → note vector → UMAP pipeline working and stable.

Week 5–6: K-Selection and Clustering

Will start running K-Means with proper initialization and multiple restarts, and selecting the optimal number of clusters using silhouette score. I will also add caching so clustering does not rerun unnecessarily. Basic tag extraction using TF-IDF will be introduced, along with centroid computation for each cluster.

Outcome: Plugin can group notes into clusters, compute centroids, and generate initial tag suggestions.

Midterm Evaluation: July 7, 2026

Week 7–8: Sidebar Panel and API Actions

Will build a sidebar panel (Joplin panel API using `setHtml()`) and connect it with the backend pipeline. The UI will display Cluster cards with editable tag names. I will implement all required Joplin API actions such as creating tags, assigning them to notes, and optionally creating notebooks.

Outcome: Users can review clusters and apply all actions from the UI.

Week 9–10: Archive, Settings, Live Indexing

Will implement archive suggestions based on simple heuristics, a settings panel for controlling parameters and re-running analysis, and live indexing by listening to note changes. I will also add LLM-based tag naming (I am thinking of supporting open AI and gemini, based on the discussion with mentors, I can implement Ollama also) and centroid-based notebook suggestions for new notes.

Outcome: Archive suggestions visible. Settings are functional. Live re-indexing works.

Week 11–12: Testing, Docs, Polish

Will test the plugin on Windows, macOS and Linux with 50, 500, 2000-note collections. I will handle edge cases which I mentioned earlier and add user documentation. Code cleanup.

Outcome: Plugin is stable, documented, and ready for submission.

Final Evaluation: August 24, 2026

5. Deliverables

- At the end of the project, I will deliver a fully functional Joplin plugin that will help users organise their notes (users will have full control).
- The plugin will allow users to analyse their entire note collection and group related notes together.
- In the plugin UI users will be able to see suggested tags for each group. They can apply them to all notes in that group with a single click or can edit according to their needs.
- The plugin will also find old, rarely edited notes and will help users identify what can be archived. This will be shown in a separate section so users can review and decide what to do.
- For better performance, the plugin will only process new or modified notes after the first run, instead of recomputing everything every time.
- The Plugin will work completely offline by default, so no note content leaves the user's device. Optional features like improved tag naming can be enabled by the user if they choose to connect to an external model.
- The plugin will also be compatible with Joplin's existing AI, so embeddings are reused and not recomputed unnecessarily.
- I will also provide documentation explaining how to install the plugin, how the features work, and what users can expect from the system.

6. Availability

Weekly availability: I can dedicate **40–50 hours per week** during GSoC and I am available for meetings or check-ins on weekends if needed.

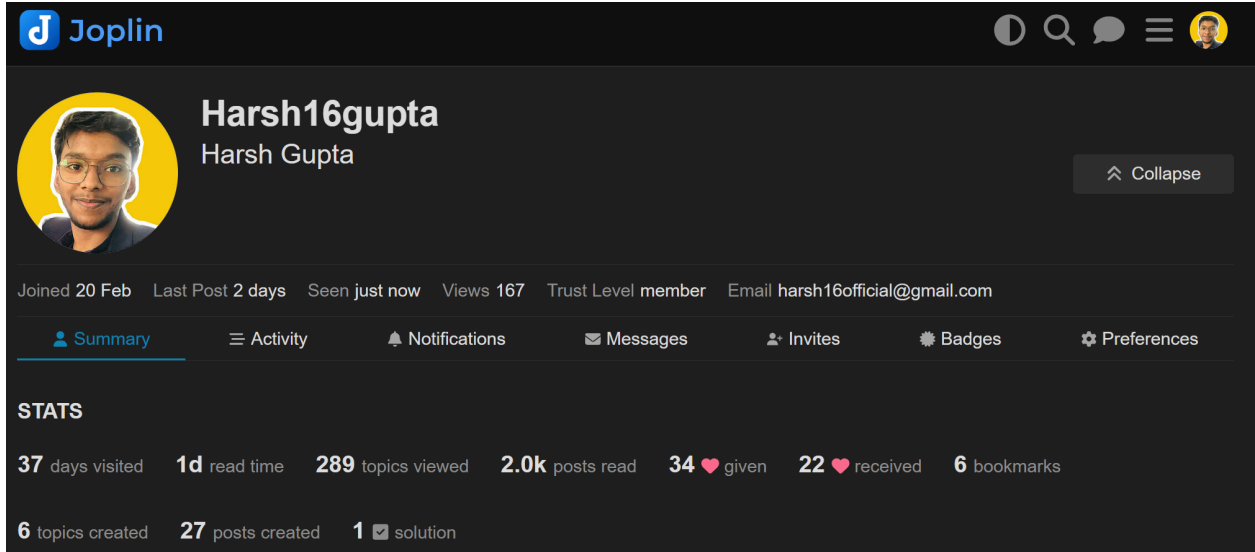
Time zone: I am in **IST (Indian Standard Time)** and flexible with scheduling calls or discussions.

Other commitments: I have my end-semester exams from May 1st to May 15th, which coincides with the community bonding period. During this time, I will be able to commit 3–4 hours per day to the project.

Communication Plan: Weekly async progress report posted to the Joplin forum thread.

Post-GSoC Plans: My goal is to make a plugin that many Joplin users can adopt in their daily workflow. After GSoC, I plan to continue maintaining the plugin by improving features, fixing issues, and responding to community feedback.

I also intend to stay active in the Joplin community by continuing to contribute to the project, helping new contributors understand the codebase and plugin architecture, and assisting in reviewing or resolving issues whenever possible.



The screenshot shows a user profile on the Joplin platform. The header includes the Joplin logo and navigation icons. The profile section features a circular profile picture of a man with glasses, the username 'Harsh16gupta', and the name 'Harsh Gupta'. A 'Collapse' button is visible. Below the profile, a row of statistics is displayed: 'Joined 20 Feb', 'Last Post 2 days', 'Seen just now', 'Views 167', 'Trust Level member', and 'Email harsh16official@gmail.com'. A horizontal menu contains links for 'Summary', 'Activity', 'Notifications', 'Messages', 'Invites', 'Badges', and 'Preferences'. The 'Summary' section is expanded, showing 'STATS' with the following data: '37 days visited', '1d read time', '289 topics viewed', '2.0k posts read', '34 ❤️ given', '22 ❤️ received', '6 bookmarks', '6 topics created', '27 posts created', and '1 📝 solution'.

I have already been actively engaged in the community through discussions on the forum and by exploring existing threads in depth. As reflected in my activity, I have consistently followed conversations, read a significant number of posts, and stayed up to date with ongoing developments.

Conclusion

I have provided a detailed overview of the project and my execution plan. My goal for GSoC 2026 is to deepen my understanding of the project while delivering meaningful features. I am familiar with the required technology stack and confident in completing the planned work within the timeline. **I am fully committed to [Joplin](#) and will not be submitting proposals to other organizations.**

Thank you for your time and consideration.